My open source Git Training with hands-on for GitHub User Group

Deep inspection of basic commands

Ilya Rokhkin

Git – meaning?

What does the word git mean?

About myself: Ilya Rokhkin – DevOps engineer

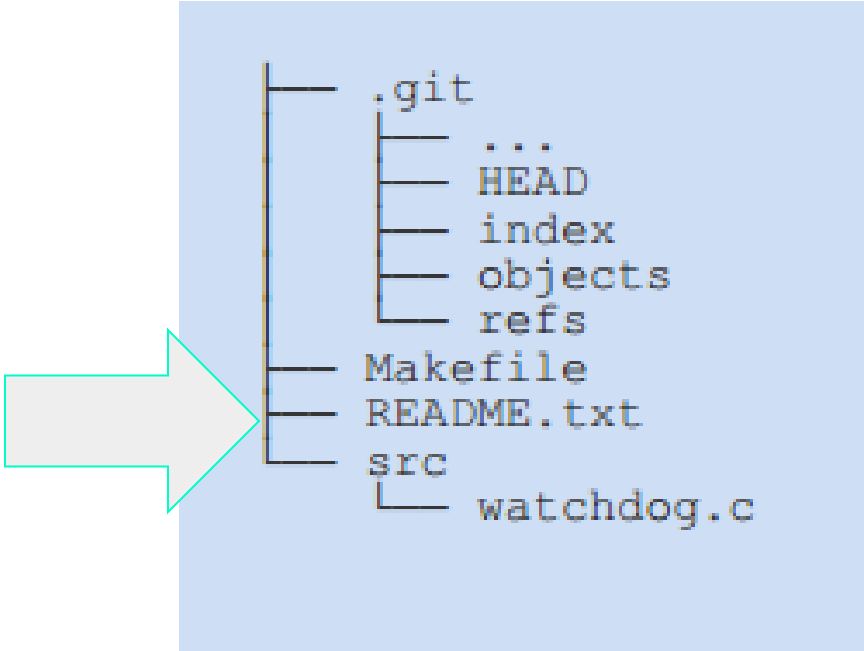
- 20+ years experience in Version Control Systems
- Git trainer in Checkpoint, Intel, Marvell, and Freelancer.
- Volunteer as a Hebrew teacher in “Ulpan” for 10+ years

The Working Tree

- The working tree has all files and folders as found in your HEAD, plus the changes you made since your last commit

- There is only ONE main working tree per repository (and only 1 .git folder as well)

The Working Tree



Objects

Every object in GIT composed of those elements –

Type – “blob”, “tree”, “commit”, “refs” (branch/tag).

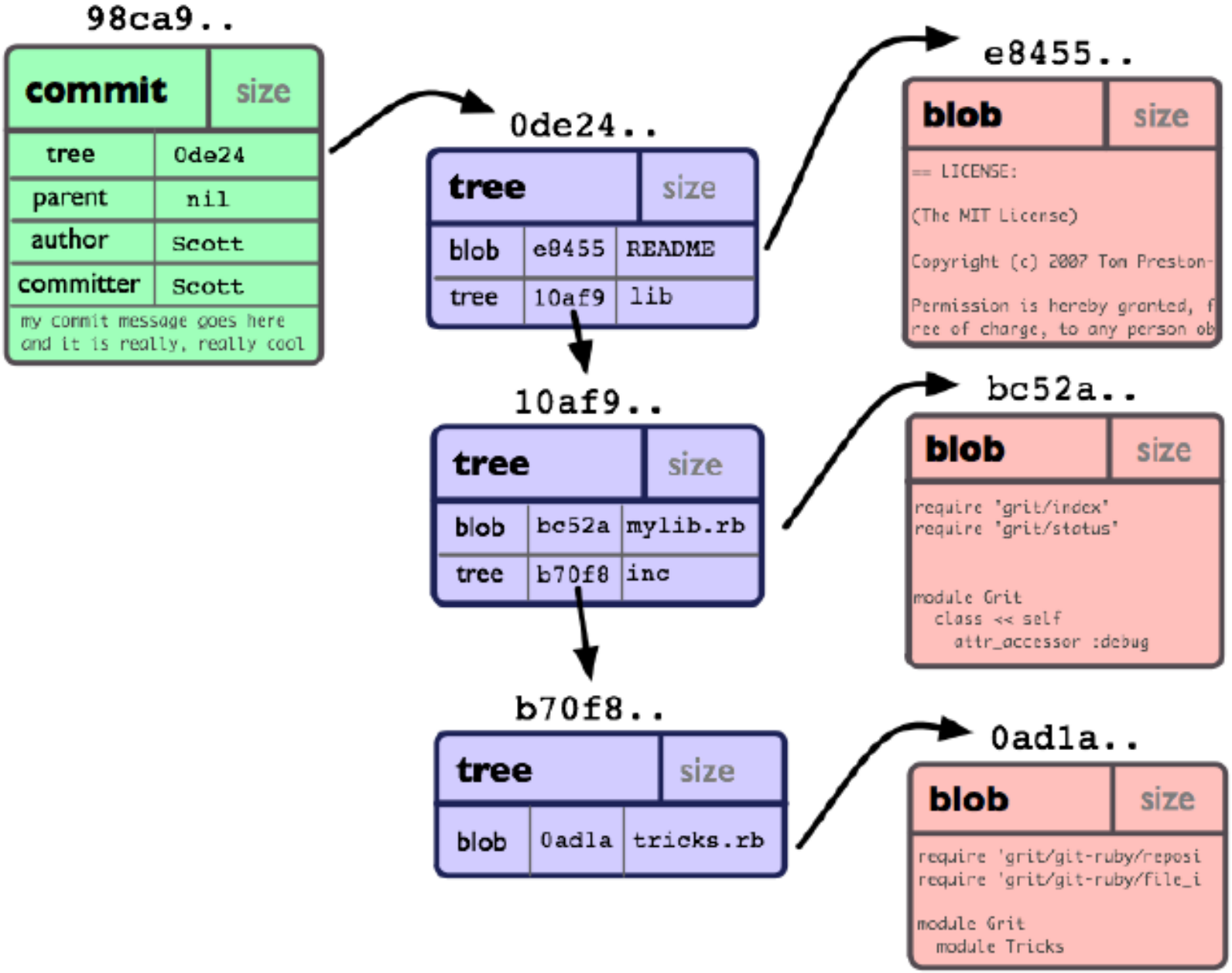
A "blob" is basically like a file – it is used to store the content of a source file.

A "tree" is basically like a directory - it references a group of other trees (subdirectories) and/or blobs (files).

A "commit" points to a single tree, marking it as what the project looked like at a certain point in time. Keeps changed files since the last commit, author of the changes, a reference to the parent commit(s), etc.

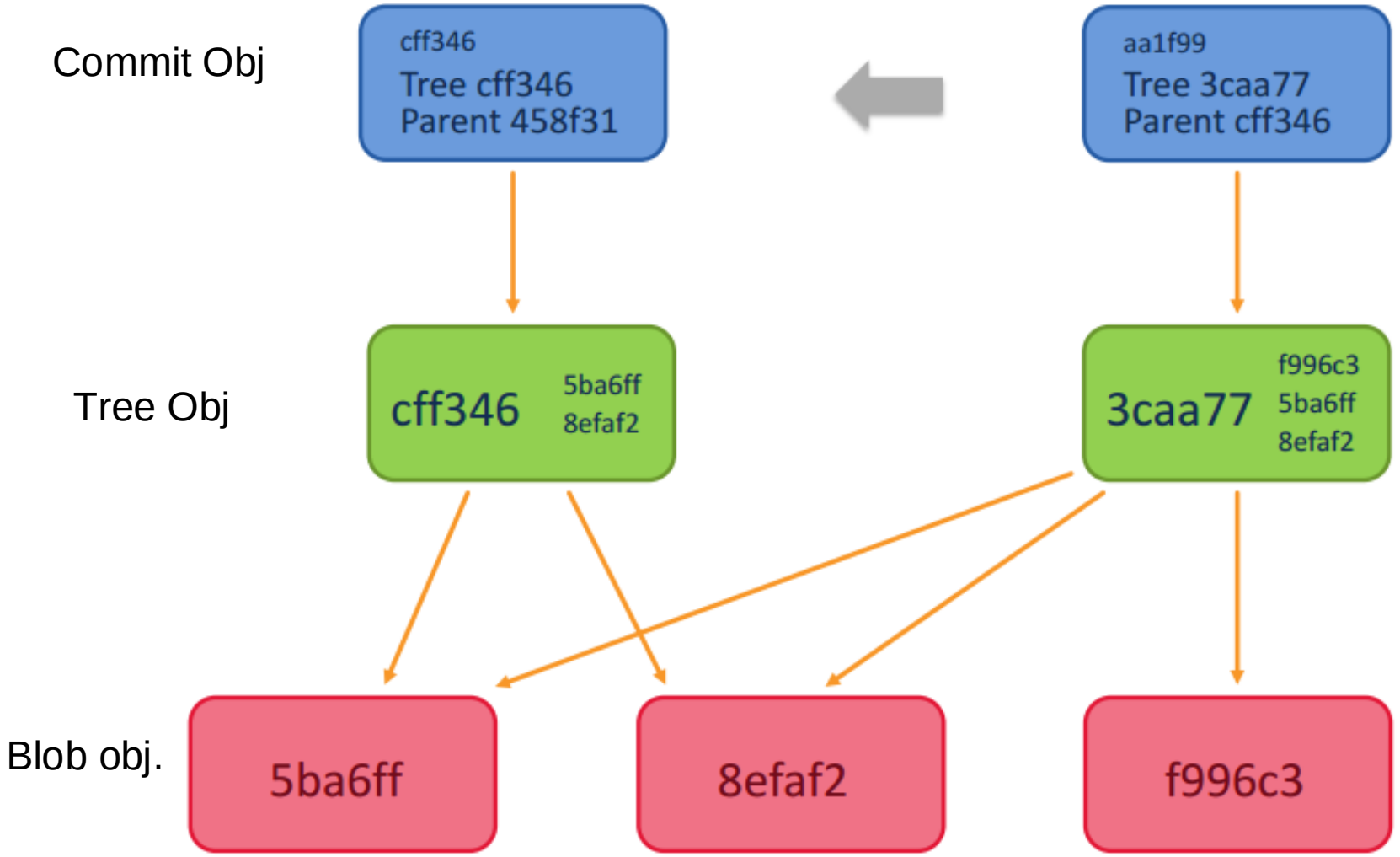
A “refs" is a way to mark a specific commit as special in some way. It is usually used to tag certain commits as specific releases or something along those lines.

Commit Object

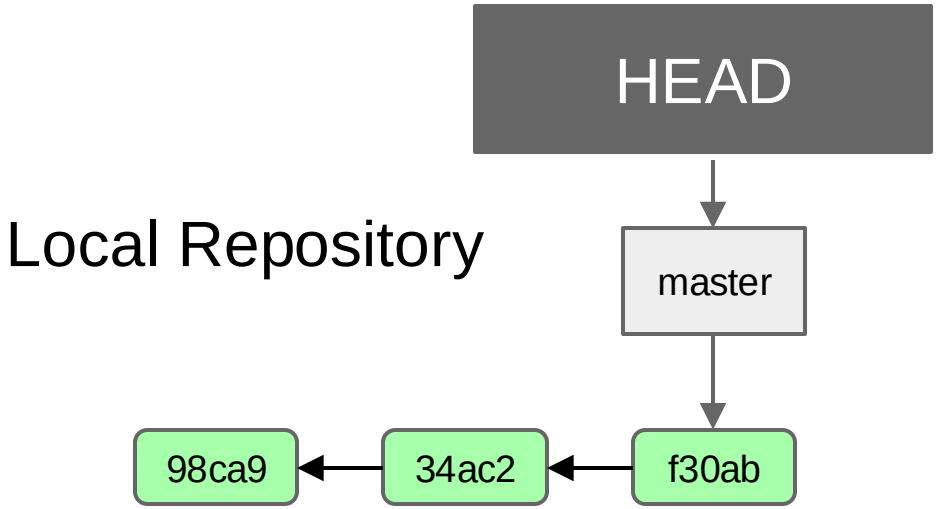


Commit object with its parent

- Links from tree object to common blobs

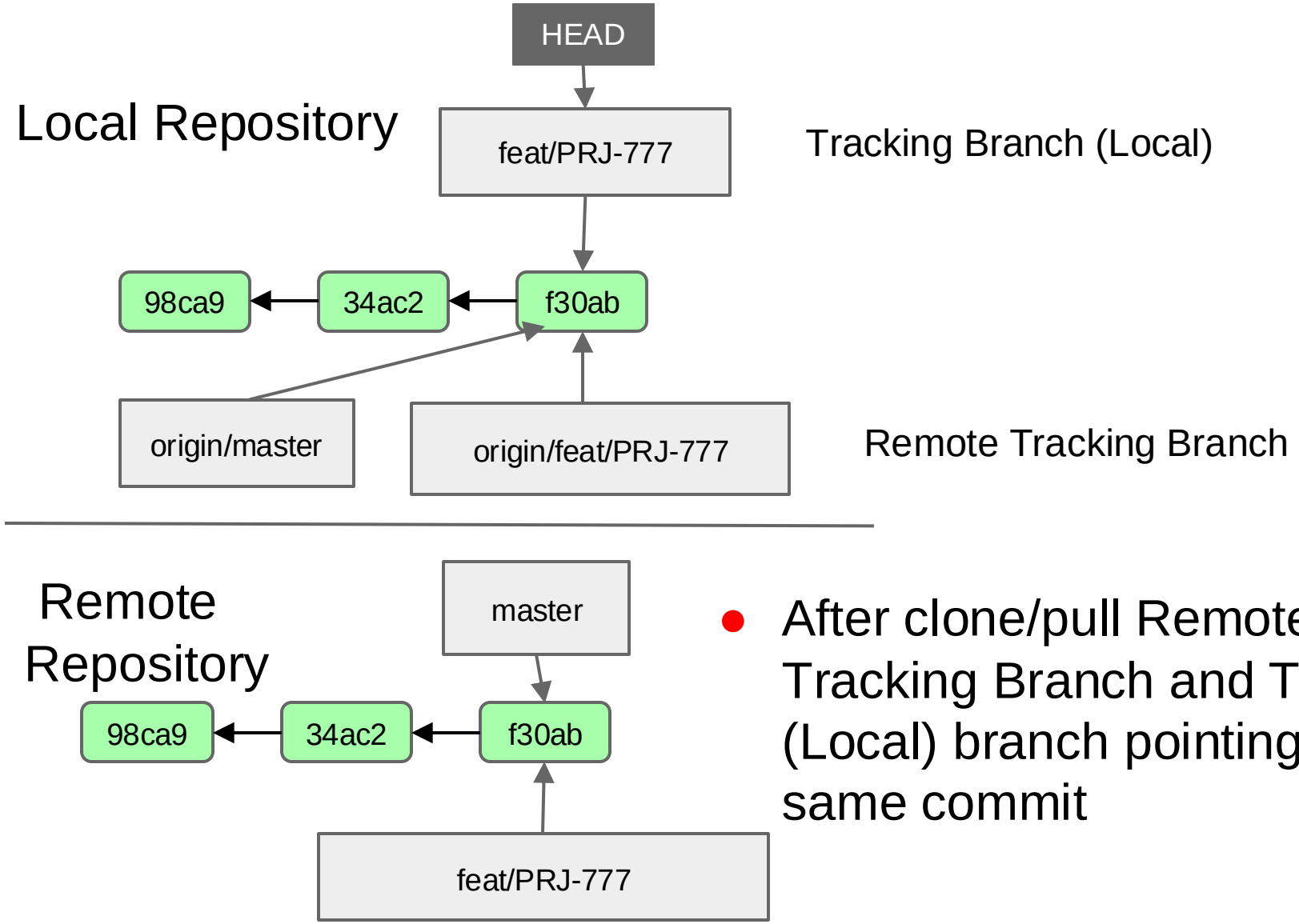


The HEAD



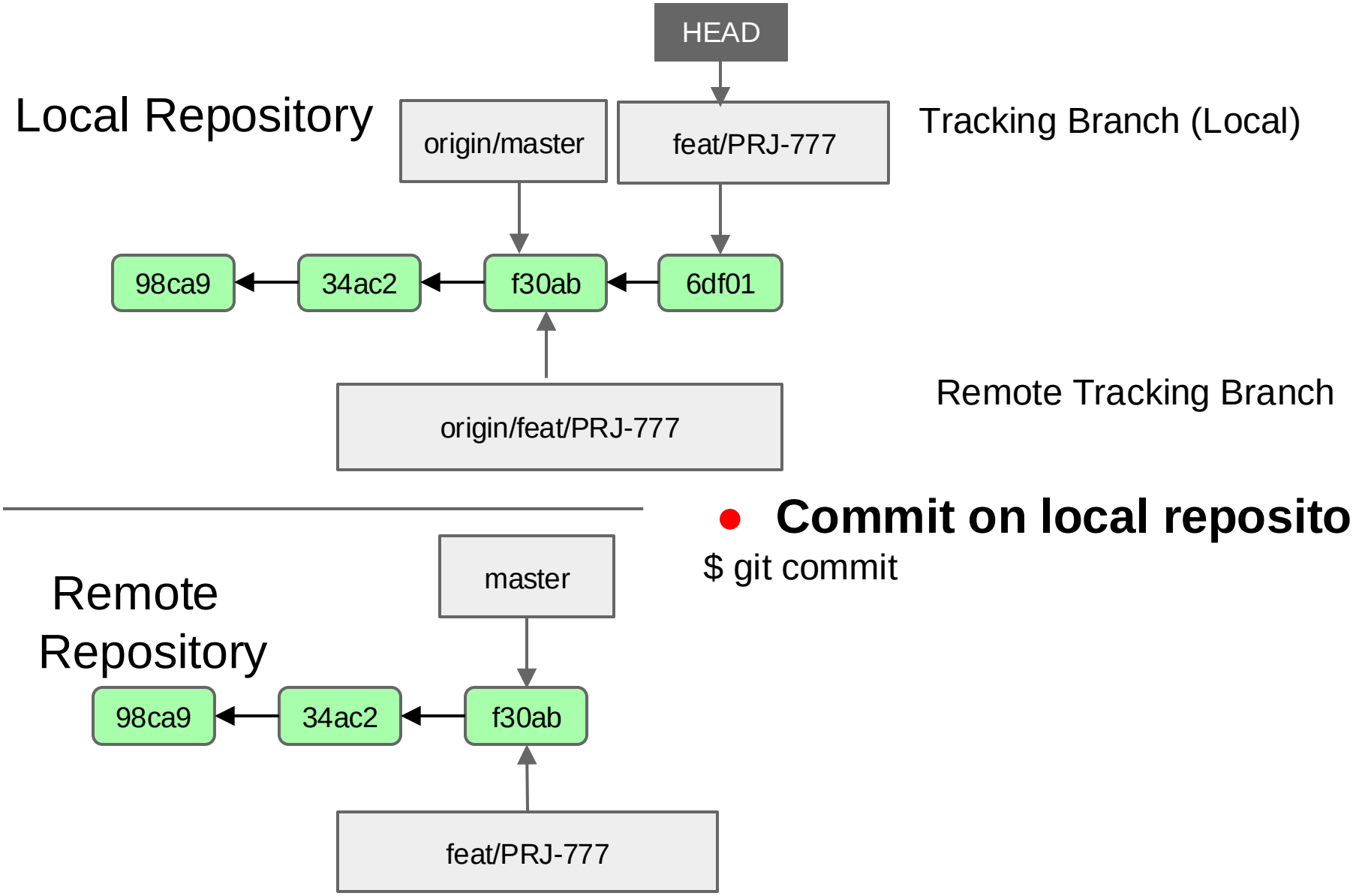
- `HEAD` is a 'pointer' to the tip of the currently checked out branch
  - In a *detached HEAD* state, `HEAD` points directly to a commit
- Only one `HEAD` per repository

Rebasing Remote Tracking Branch



- After clone/pull Remote Tracking Branch and Tracking (Local) branch pointing to the same commit

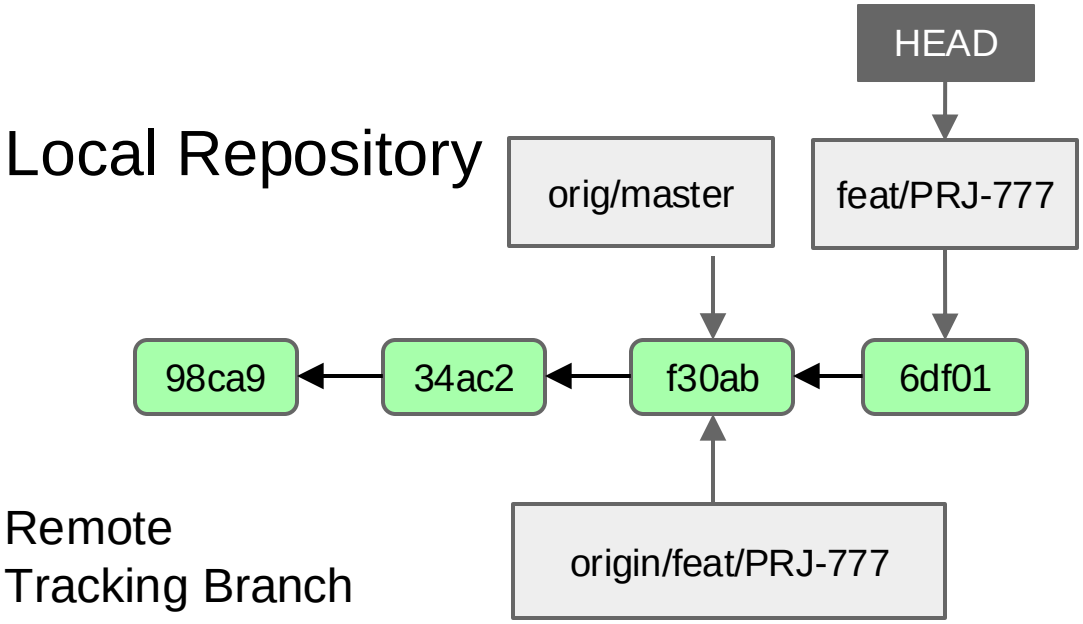
Rebasing Remote Tracking Branch

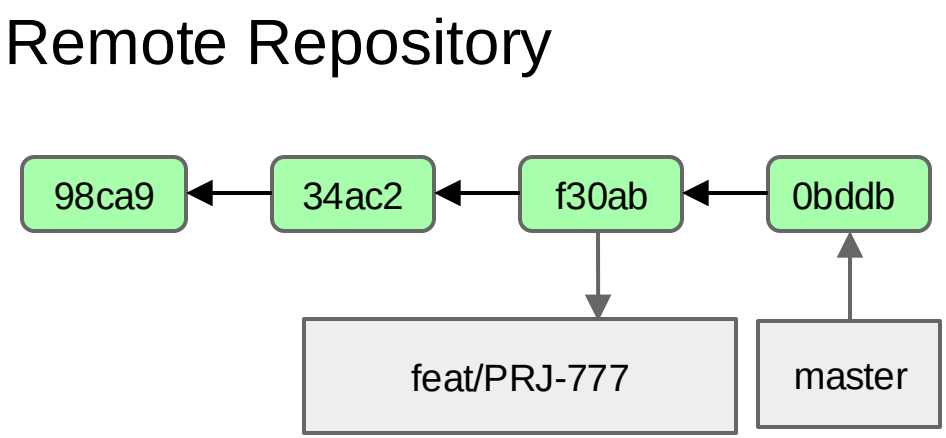


- **Commit on local repository**

$ git commit

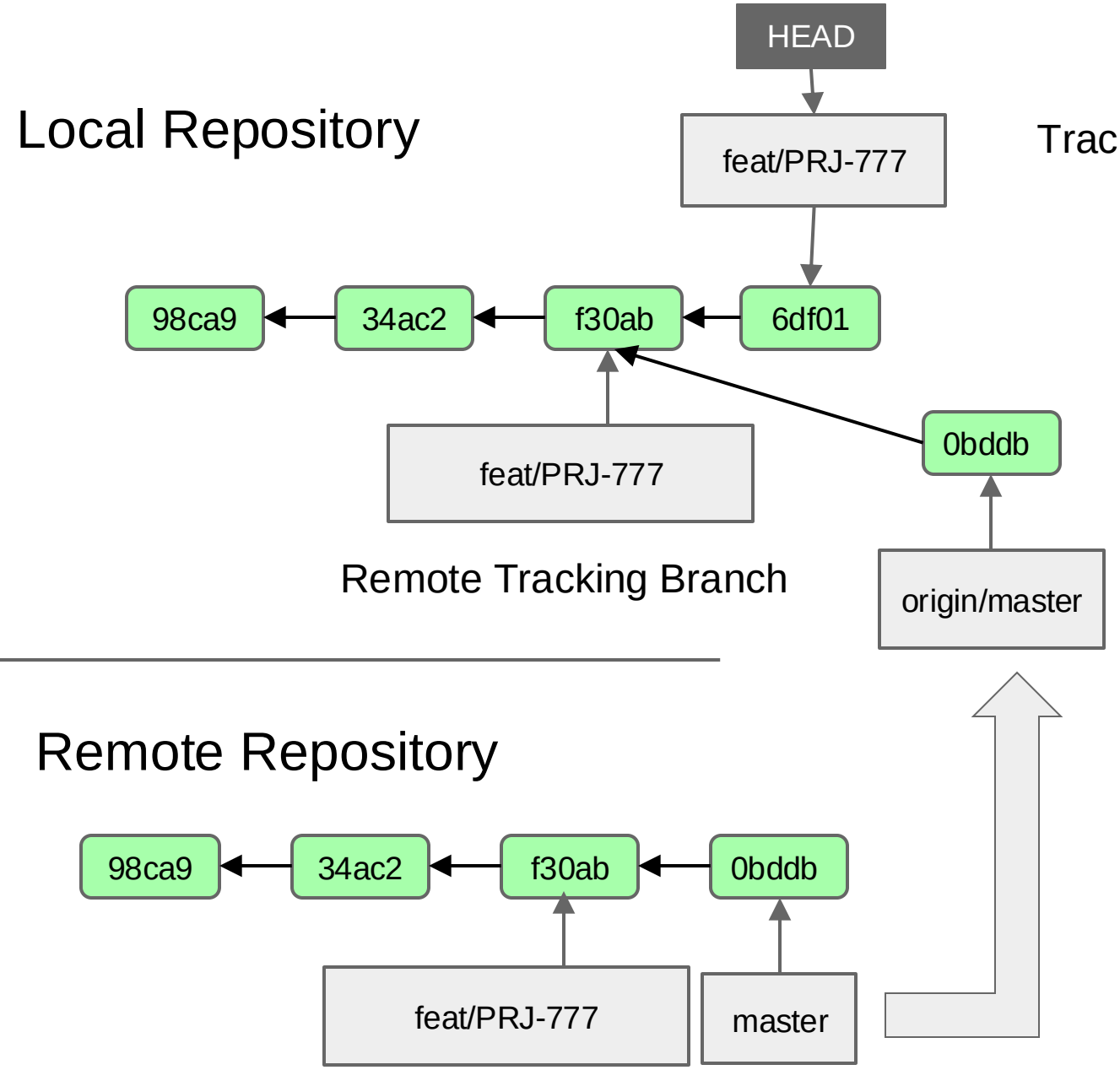
Rebasing Remote Tracking Branch





- **Merge another Commit to remote repository from another feature branch**

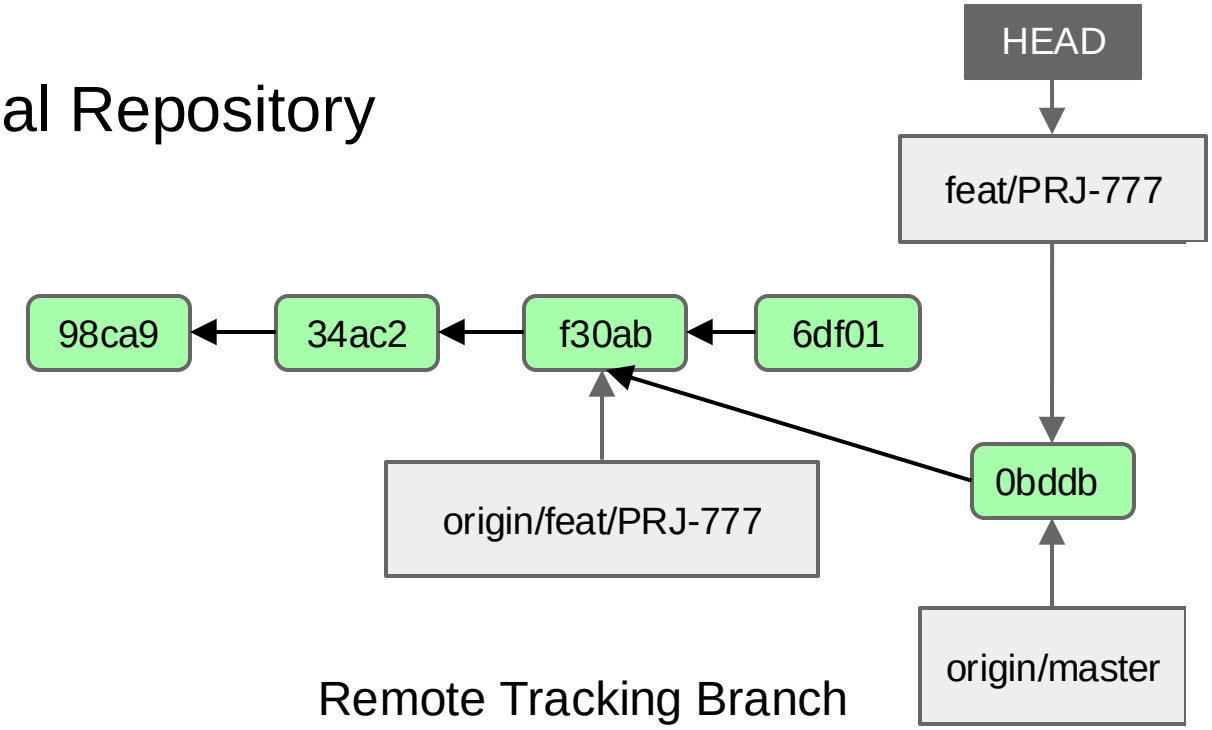
Rebasing Remote Tracking Branch



- **Fetch/Pull the Commit from remote repository to Remote Tracking Branch**

Rebasing origin/master

Local Repository



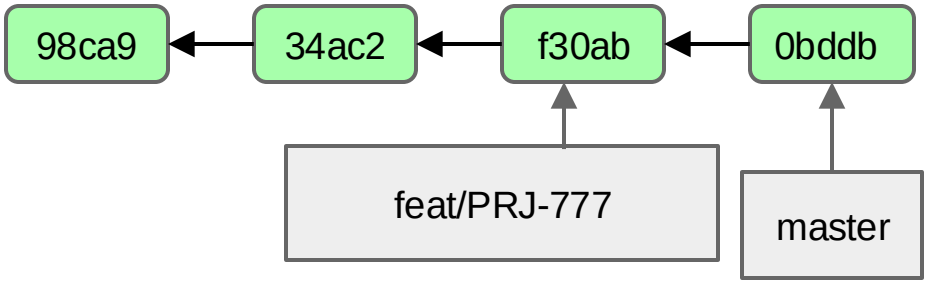
Tracking Branch (Local)

Remote Tracking Branch

- $ git rebase origin/master

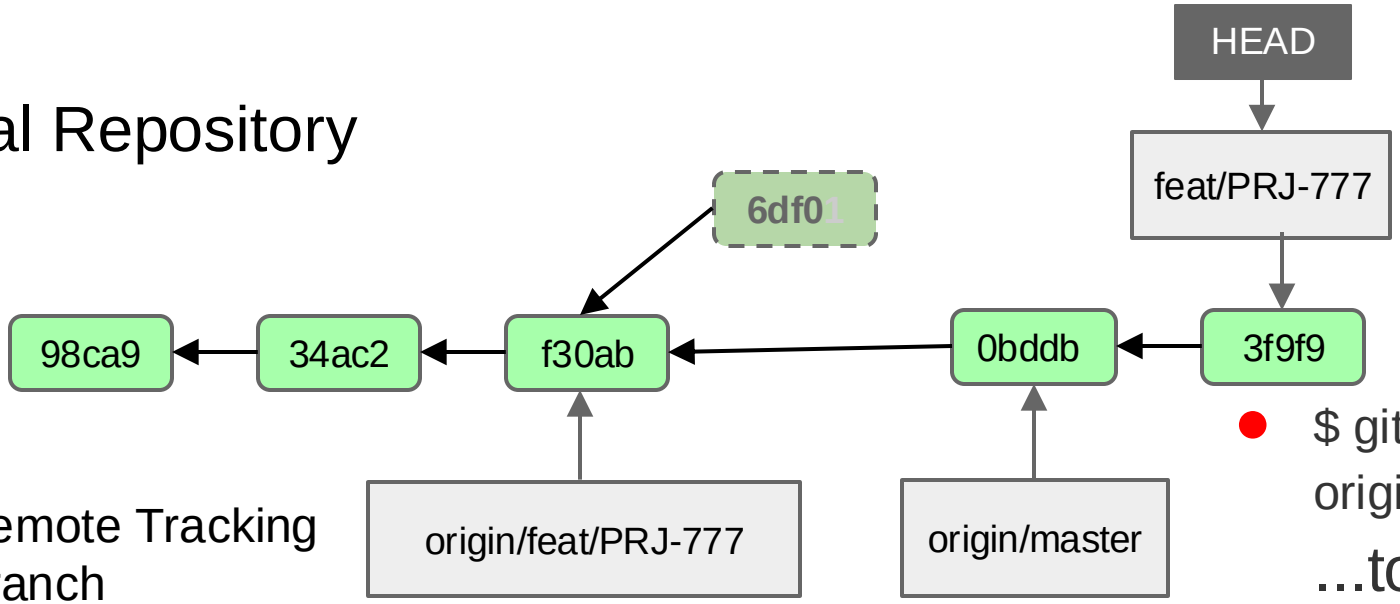
First, rewinding head to replay your work on top of it...

Remote Repository



Rebasing Remote Tracking Branch

Local Repository



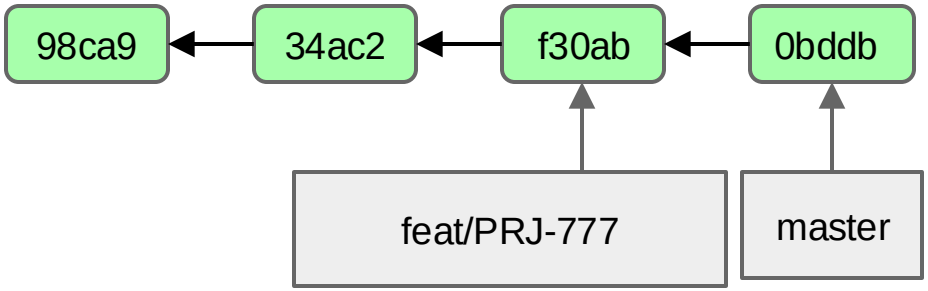
Tracking Branch (Local)

Remote Tracking Branch

- $ git rebase origin/master

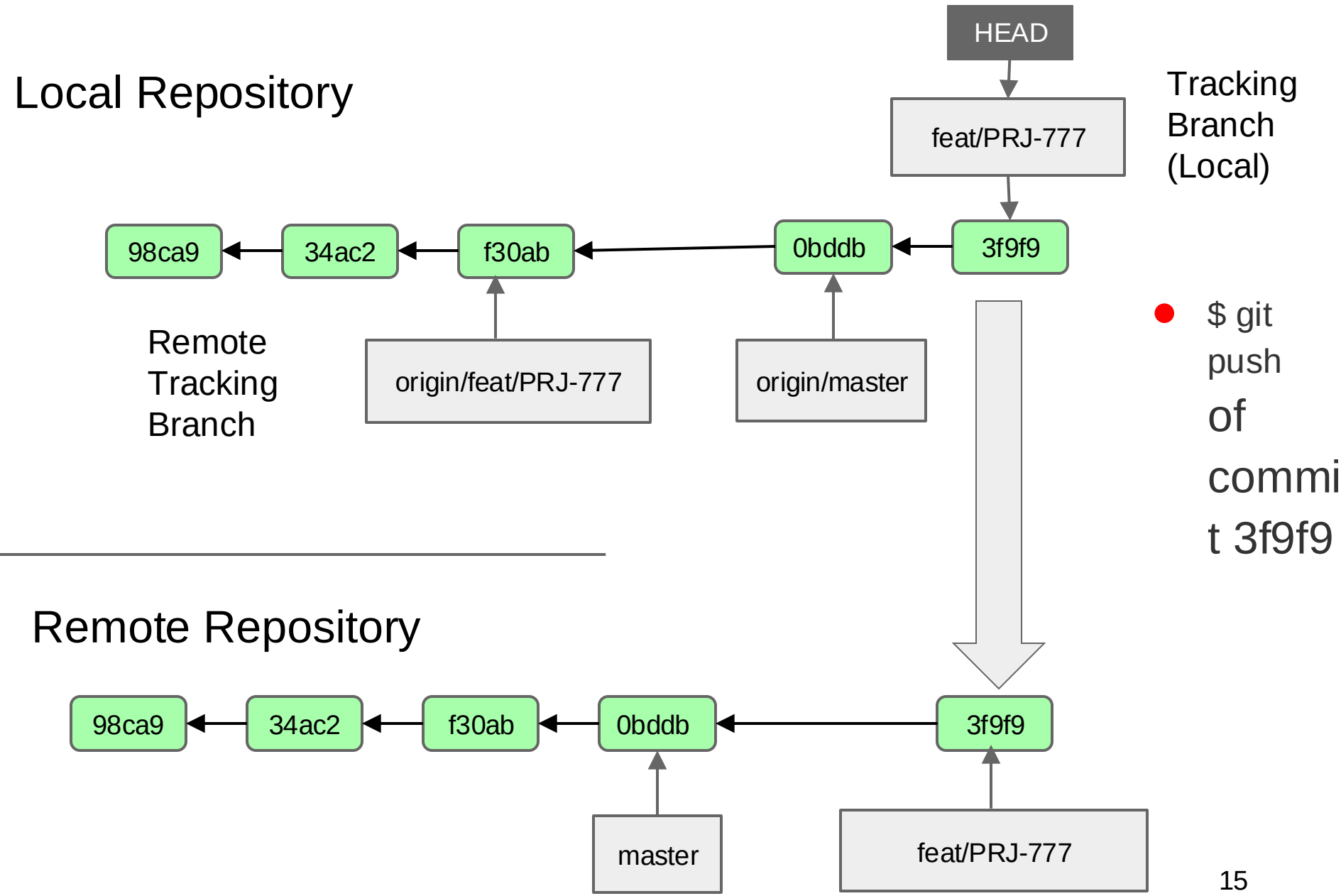
...to replay your work on top of it…

Remote Repository



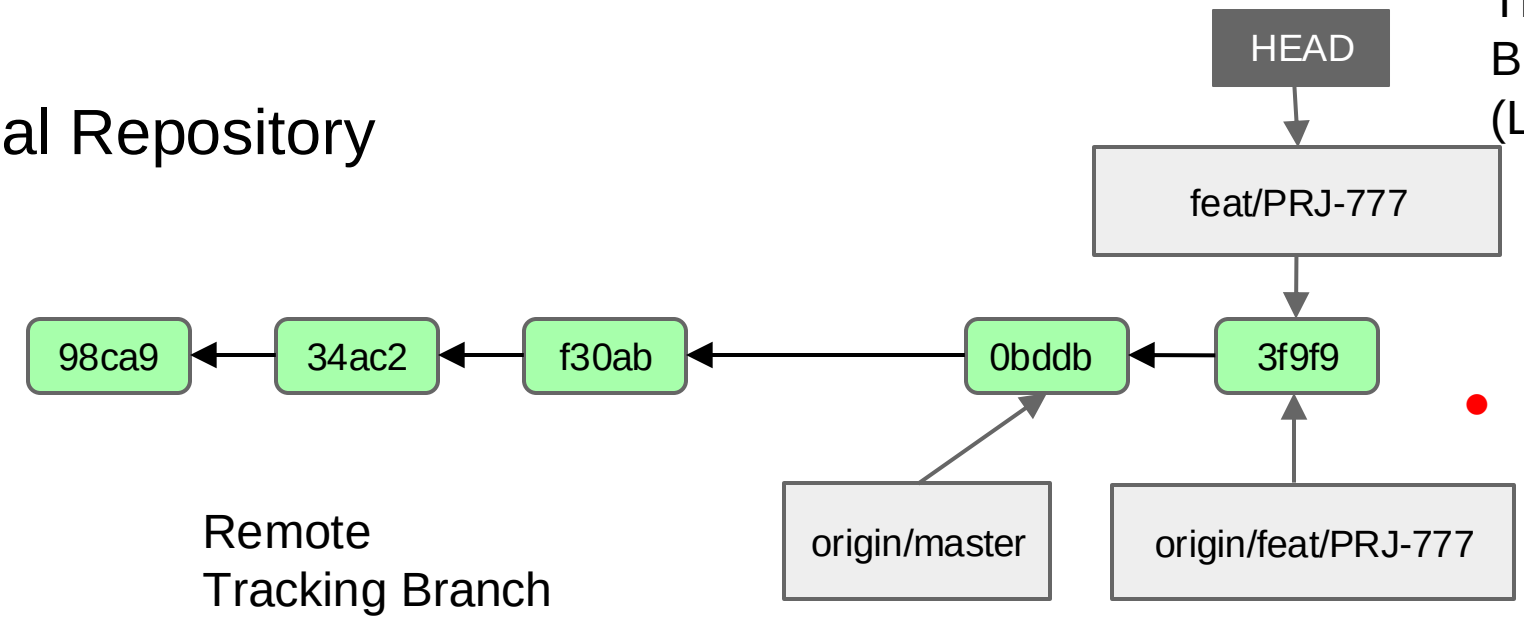
Applying: diff/patch of commit 6dfo on top of origin/master to create commit 3f9f9

Rebasing Remote Tracking Branch



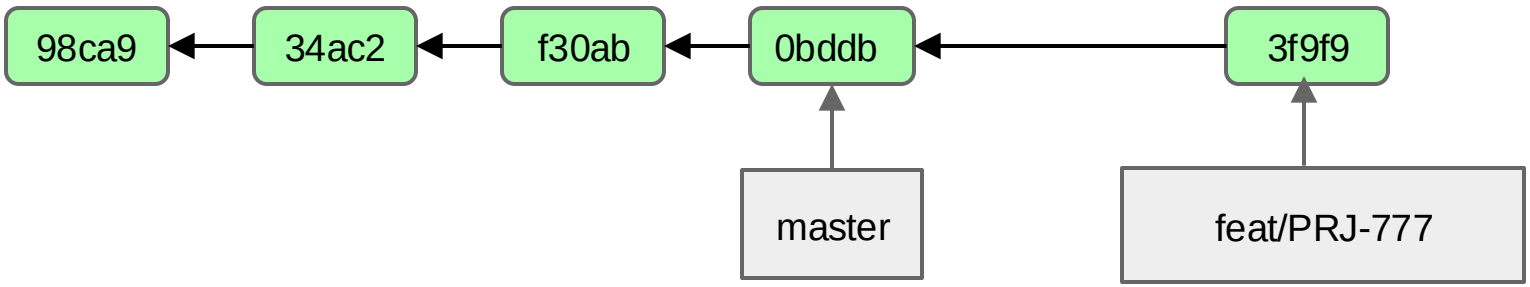
Rebasing Remote Tracking Branch

Local Repository



- After push Remote Tracking Branch and Tracking (Local) branch points to the same commit

Remote Repository



Perils of rebase

- **Do not rebase commits that you have pushed to a public repository.**

If you follow that guideline, you'll be fine. If you don't, people will hate you, and you'll be scorned by friends and family.

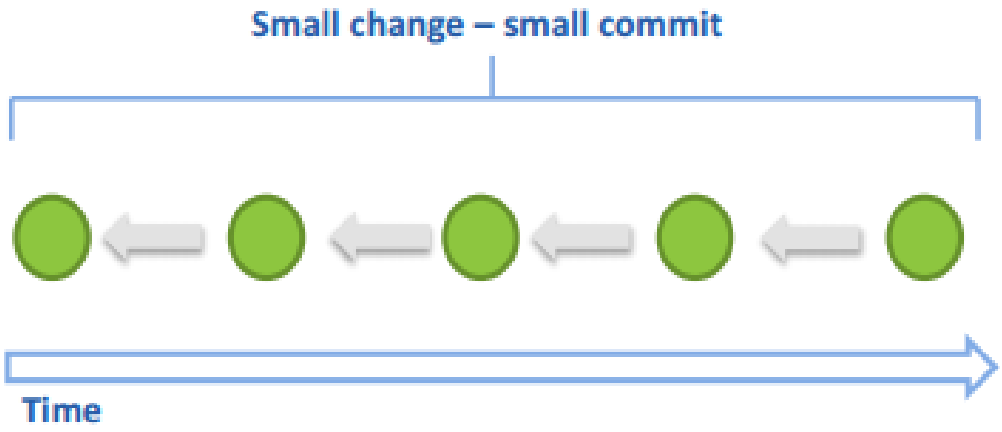
Question?

Thanks!

Backup Slides

Best practices – commit

Keep changes small wherever possible and commit frequently



Best practices – squashing

Before pushing, squash related changes together to make for better understanding by others

Best practices - concise commit messages

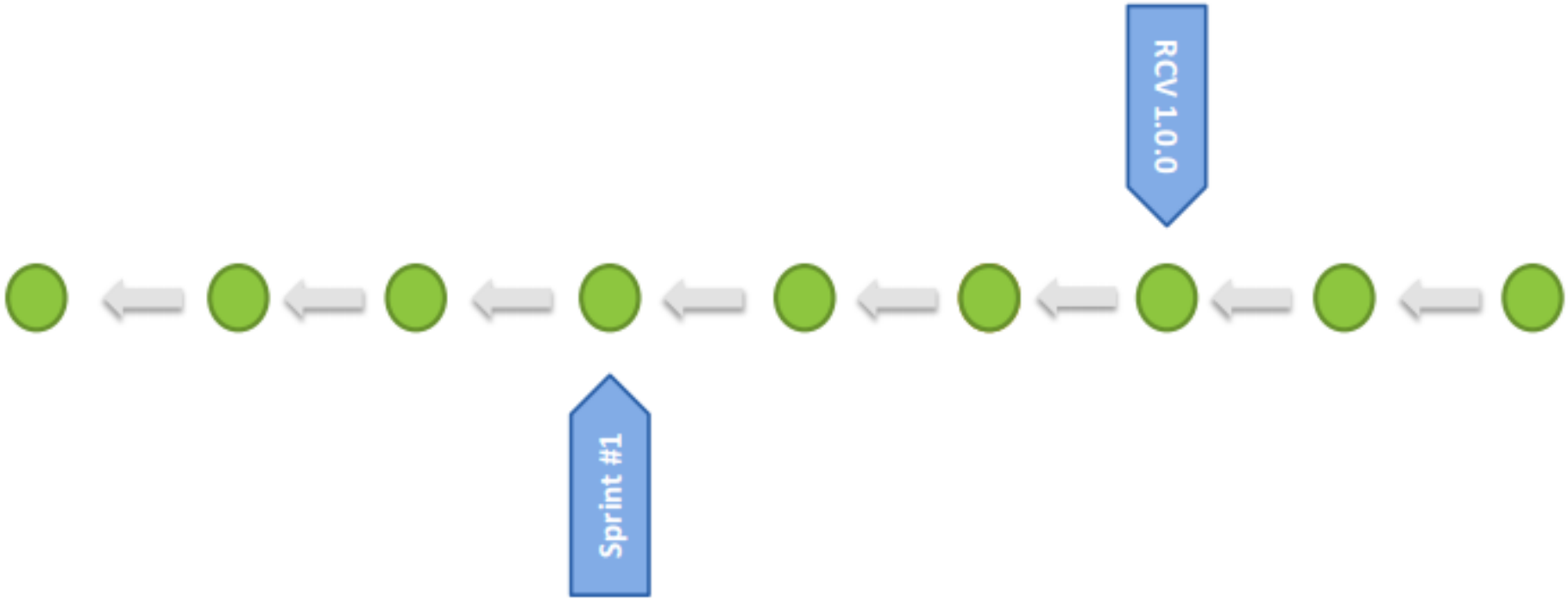
Limit commit message header to **60 characters** and add the meaningful details in the rest of the message

```
commit f6ce5cc010bf6665a8f2a701e7983e0c2ac8f144
Author: Shawn O. Pierce <sop@google.com>
Date:   Thu Nov 29 09:55:47 2012 -0800

    Sort comments before emailing them

    The order supplied by the caller can be random, ensure comments
    get sorted into a sane order before they are included into the email.

    Bug: issue 1692
    Change-Id: ibd85e514977545d022f936a5993f2a6ef6e52321
```

Best practices – tag milestones

Tag important milestones (for history and for accessibility)